

TELECOM SELF-SERVICE & E-COMMERCE SYSTEM

Comprehensive Project Blueprint, Architecture Spec & Implementation Roadmap

Document Type: Full Lifecycle Technical Specification

Target Architecture: Python (Django) / React Native (Expo) / PostgreSQL

Scope: SIM Lifecycle, National ID e-KYC, Device E-Commerce, Digital Payments

Status: Production Ready / Implementation Baseline

1. Executive Summary & Vision

In modern telecommunications ecosystems, standard customer operations—such as registering a new SIM card, changing profile ownership info, correcting personal data errors, blocking services due to hardware loss, and purchasing cellular devices—traditionally require consumers to physically visit regional branch hubs. This creates major operational friction, long counter queues, increased overhead costs for telecom operators, and significant service delays.

This blueprint details the implementation plan for an end-to-end **Telecom Self-Service & E-Commerce System**. The objective is to transition physical branch workflows into a unified, secure digital platform. Using web and cross-platform mobile portals, customers can safely execute lifecycle transactions, verify profiles against official National ID authorities, modify registry discrepancies, order packages, and purchase hardware.

Strategic System Objective: To build a scalable, highly secure environment that replicates telecom core switches and third-party validation networks using mock structural APIs. This delivers an immediate Minimum Viable Product (MVP) ready for pilot deployment.

2. System Scope & Functional Architecture

The system is organized into four distinct functional modules, each managing a specific segment of the telecom operations domain.

2.1. SIM Lifecycle Management Module

- **Instant SIM Loss & Service Suspension:** Allows users to immediately lock a SIM card and pause billing/network access if a phone is lost or stolen. This protects against cellular fraud.
- **SIM Replacement & Cloning Approvals:** A controlled process to order a replacement SIM card retaining the previous MSISDN (phone number), protected by multi-step ownership verification.
- **Digital Activation & Number Selection:** Allows new users to search an inventory of available mobile numbers, upload required pictures, select pricing tiers, and trigger provisioning.

2.2. Identity Management & National ID e-KYC

- **National ID Registry Verification:** Connects customer accounts to their legal records using an internal electronic Know-Your-Customer (e-KYC) matching matrix.
- **Data Correction Workflows:** Provides a secure pipeline for users to upload documentation when correcting typos, names, or birth dates on old telecom profiles, sending them directly to back-office workers for review.

2.3. E-Commerce & Inventory Pipeline ("Tele Shop")

- **Advanced Device Cataloging:** Supports rich text, dynamic media, storage specs, network band compatibility, and warranty info for mobile devices.
- **Order Fulfillment Logic:** Tracks order statuses through an interactive system (e.g., Pending → Processing → Out for Delivery → Delivered).

2.4. Package Subscriptions & Payment Orchestration

- **USSD Substitution Engine:** Moves voice, data, and value-added services (VAS) out of classic, clunky USSD menus (e.g., *999#) into clean, visual interfaces.
- **Payment Gateway Interfaces:** Implements webhook listeners and secure redirection workflows to handle local mobile wallets (such as Telebirr) and bank card APIs.

3. Technology Stack Selection Matrix

LAYER	SELECTED TECHNOLOGY	ARCHITECTURAL RATIONALE
Backend Framework	Django 5.x / DRF	Provides an auto-generated, highly customizable back-office Admin interface for telecom employees right out of the box. Includes built-in protection against common security vulnerabilities.
Mobile Frontend	React Native + Expo	Allows building a single JavaScript/TypeScript codebase that runs natively on both Android and iOS devices. Provides direct access to device cameras for easy ID scanning.
Database Engine	PostgreSQL 16	A reliable relational database that enforces strict ACID compliance for transactional processing, audit logs, and complex structural relationships.
Asynchronous Tasks	Celery + Redis	Handles background network actions without blocking users, such as sending confirmation SMS alerts and polling third-party payment statuses.

4. Relational Database Schema Design

The relational database model must maintain strict structural references to ensure zero-loss tracking across profiles, telecom hardware assets, and financials.

TABLE: users_profile

PK id: UUID
phone_number: VARCHAR(15) [Unique, Nullable]
email: VARCHAR(255) [Unique]
national_id_number: VARCHAR(50) [Unique, Nullable]
is_verified_kyc: BOOLEAN [Default: False]
date_joined: TIMESTAMP

TABLE: sim_cards

PK id: BIGINT
FK user_id: UUID → users_profile(id)
msisdn: VARCHAR(15) [Unique] (The Mobile Number)
imsi: VARCHAR(15) [Unique] (International Mobile Subscriber Identity)
iccid: VARCHAR(20) [Unique] (SIM Card Serial Number)
status: VARCHAR(20) [Choices: ACTIVE, SUSPENDED, BLOCKED, PENDING_ACTIVATION]
updated_at: TIMESTAMP

TABLE: service_requests

PK id: BIGINT
FK user_id: UUID → users_profile(id)
request_type: VARCHAR(30) [Choices: SIM_BLOCK, SIM_REPLACEMENT, DATA_CORRECTION]
payload_data: JSONB [Stores dynamic form fields & documents]
status: VARCHAR(20) [Choices: SUBMITTED, UNDER_REVIEW, APPROVED, REJECTED]
FK reviewed_by: INTEGER [Nullable] → django_admin_user(id)
admin_notes: TEXT

TABLE: device_inventory

```
PK id: BIGINT
brand: VARCHAR(50)
model_name: VARCHAR(100)
specifications: JSONB [RAM, Storage, Battery, Camera details]
price: DECIMAL(12, 2)
stock_quantity: INTEGER
```

5. Third-Party Integration Framework (Mock Layer)

Because building production-level telecom applications requires direct network integration, this project uses a highly reliable mock abstraction layer to mimic real-world APIs.

5.1. National ID Agency API Simulation

The backend sets up isolated mock endpoints that replicate official government lookups. When a user submits their National ID, the backend acts as a proxy:

```
POST /api/v1/mock-gov/validate-id/
Payload: { "id_number": "NID-8832-ETH", "full_name": "..."}
Response: { "matched": true, "citizenship_status": "VERIFIED" }
```

5.2. Telebirr Payment Engine Integration Workflow

To process transactions without a live merchant token, the application implements the standard digital wallet checkout lifecycle using clean backend endpoints:

1. The client app initiates an order and requests a payment token from Django.
2. The Django server creates an internal invoice record marked as PENDING and generates a secure checkout redirect link pointing to a built-in checkout testing template.
3. The simulated template includes a button that sends an automated, signed webhook alert (POST /api/v1/payments/telebirr-webhook/) back to the server to safely verify the transaction.
4. The backend validates the webhook payload signature, marks the transaction as SUCCESSFUL, and updates the device inventory or unlocks the requested SIM tier.

6. Implementation Timeline & Roadmap

The execution schedule is split across four specific phases to ensure steady development and thorough testing of all core features.

Phase 1: Foundation & Core Database Configuration (Weeks 1-2)

- Initialize the core Django workspace and configure connections to PostgreSQL.
- Deploy authentication structures using Django REST Framework and configure JWT token handling.
- Build the foundational schema layout and run initial migrations to prepare the database.

Phase 2: Admin Dashboard & Workflow Orchestration (Weeks 3-4)

- Build out the core Django admin panel with customized filters for managing SIM statuses and service logs.
- Implement file handling for document uploads, allowing users to safely submit National ID images.
- Create the business logic for state transitions, ensuring only authorized team members can approve changes.

Phase 3: Cross-Platform UI Development (Weeks 5-6)

- Set up the React Native mobile project workspace using the Expo CLI toolset.
- Design intuitive screens for the e-KYC entry portal, the self-service dashboard, and the phone catalog.
- Integrate Axios client libraries to manage API connections and authentication headers smoothly.

Phase 4: Integration, Testing & Validation (Weeks 7-8)

- Build the mock payment webhooks and run edge-case tests on checkout pipelines.
- Verify security controls, checking that regular users cannot access administrative endpoints.
- Optimize SQL queries and execute end-to-end user tests to finalize the system for launch.

7. Comprehensive System Security Strategy

Security Warning: Telecom applications are frequent targets for identity theft and fraudulent SIM-swapping attacks. The system must enforce security checks at every level of the application stack.

- **Role-Based Access Controls (RBAC):** Administrative control endpoints must be protected by explicit permission checks. The system ensures that regular users cannot view or edit pending service requests.

- **Immutable Security Audit Logging:** The database captures comprehensive tracking data for every critical administrative action—logging the exact worker ID, timestamp, and field values whenever a SIM card status is modified.
- **Secure Document Processing:** Uploaded images and identification documents are saved to private folders with randomized filenames, keeping user data safe from public access.